

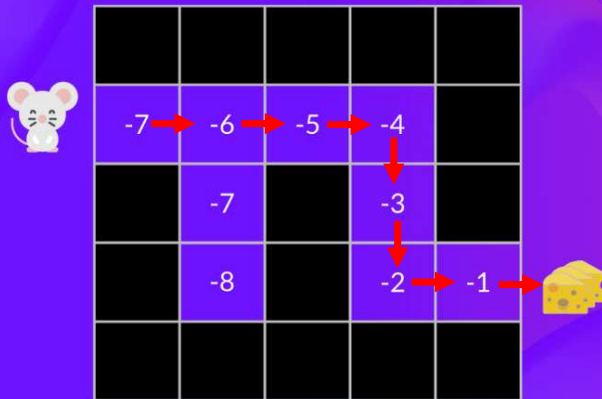
Reinforcement Learning

Naf Guo
2026

价值函数

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value
function

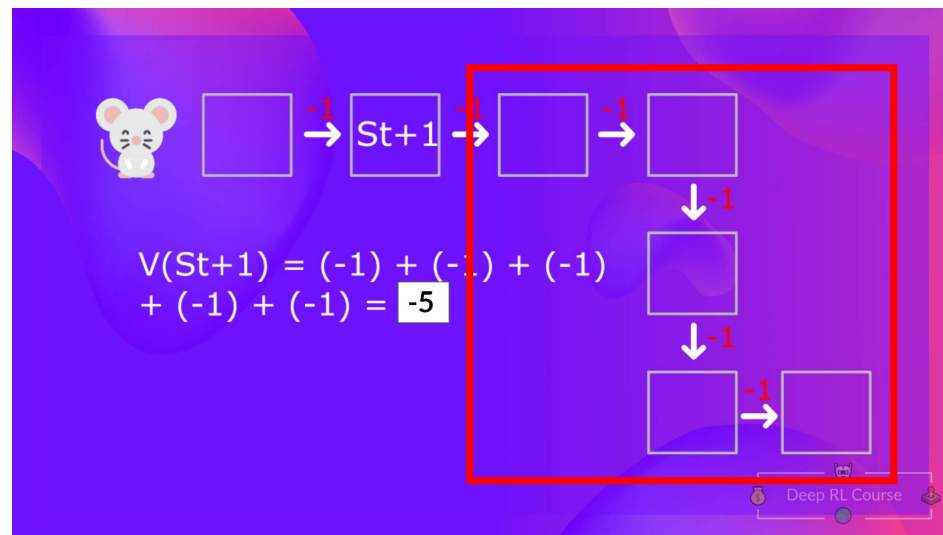
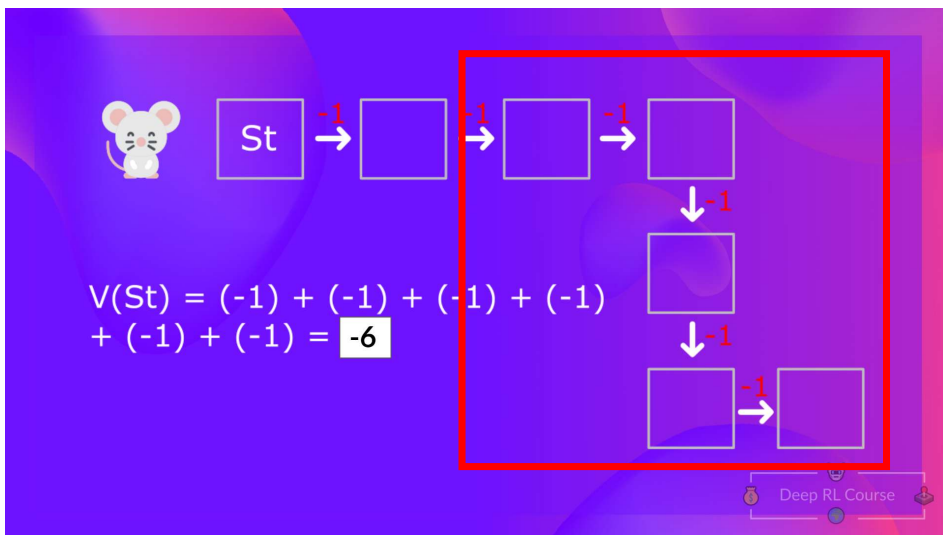
Expected discounted return

Starting
at state s

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

上一次，由于面对的条件比较简单，对每一个位置，我们都穷举地找到了到达奶酪处的最短路径，并依据此计算得到了每个状态的价值函数

贝尔曼方程 Bellman Equation



在上面小老鼠偷奶酪的例子中，为了算 S_t 处的价值函数，我们需要按箭头走完全程；为了算 S_{t+1} 处的价值函数，我们也要按箭头走完全程。

你会发现，中间很多计算是重复的。

贝尔曼方程 Bellman Equation

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s]$$

Value function Expected discounted return Starting at state s

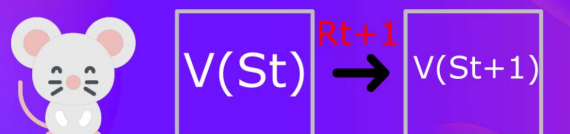
$$V(S_t) = E[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s]$$

从 S_t 走一步，获得奖励 R_{t+1} ，到达 S_{t+1}

$$V(S_{t+1}) = E[R_{t+2} + \gamma R_{t+3} + \dots | S_{t+1} = s(t+1)]$$



$$V(S_t) = R_{t+1} + \gamma V(S_{t+1})$$



$$V(S_t) = R_{t+1} + \gamma * V(S_{t+1})$$



$$V(S_t) = -1 + 1 * (-5)$$
$$V(S_t) = -6$$



贝尔曼方程 Bellman Equation

The Bellman Equation

$$V_{\pi}(s) = \mathbf{E}_{\pi} [R_{t+1} + \gamma * V_{\pi}(S_{t+1}) | S_t = s]$$

Value of
state s

Expected value of
immediate reward

+ the discounted value of
next_state

If the agent
starts at state s

And uses the policy to
choose its actions for
all time steps



$V(S_t)$

R_{t+1}

$V(S_{t+1})$

$$V(S_t) = R_{t+1} + \gamma * V(S_{t+1})$$



这个式子将是未来学习的核心

贝尔曼方程就是一句话：

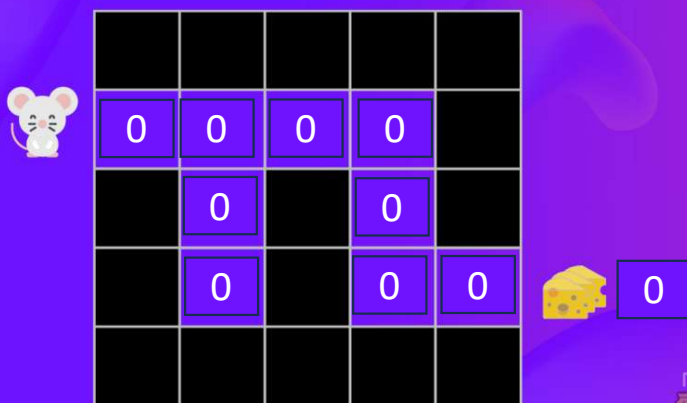
当前状态的价值 = 即时奖励 + 未来状态价值的折扣和

当前在状态 s 处，走一步会获得一个reward R_{t+1} ，到达 S_{t+1} 状态
也就是 $V(S_t) = R_{t+1} + \gamma V(S_{t+1})$

价值迭代

The State Value Function

State Value Function: calculate the value of a state.



所有状态的价值初始化为0
包括奶酪格子价值也是0

为了使得老鼠采取的行动数目尽可能小，每走一步，都在reward里加入一个-1的惩罚；作为无良的人类，我放的奶酪其实是假的，因此到达奶酪的奖励为0，但一旦到达，整个过程就结束了。

设置 $\gamma = 1$ ，也就是不折扣。

小老鼠采取贪心策略，也就是只会选择价值最大的动作。

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value
function

Expected discounted return

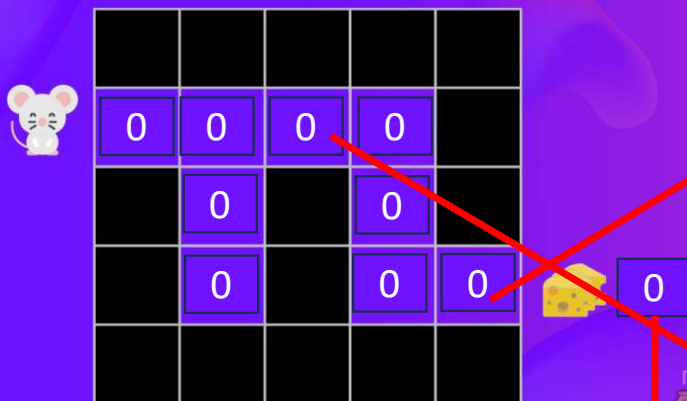
Starting
at state s

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

价值迭代: 第1轮

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

一定要注意, **return** 是从当前时刻到终止时刻的奖励总和, 因此只有过程结束后才能得到一个确定的数值。

第一轮迭代, 我们要根据初始化的价值函数+贪婪策略, 以及贝尔曼方程, 对价值函数进行更新。

考察这个格子, 它可以:

1) 向左走1步, reward为-1, 到达格子的价值打分为0, return为-1;

2) 向右走1步, reward为-1, 到达格子也是0(因为作为邪恶人类, 我放了假奶酪, 没有奖励), 则return为-1;

两种做法return均为-1, 根据贪婪策略都可以走, 并且更新此位置价值函数为-1.

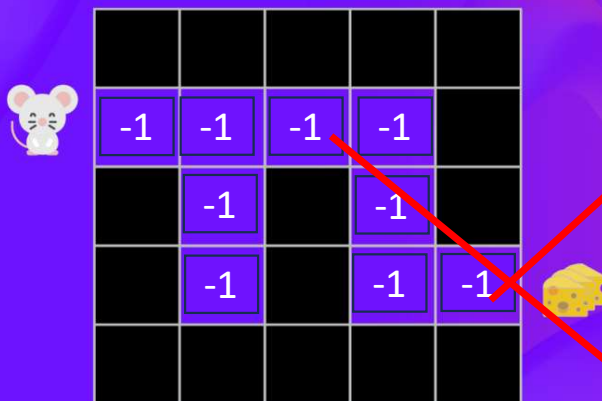
对任意格子, 第一轮迭代的情况是一样的, 最后都会使价值函数更新为-1

注意奶酪格子是所谓吸收态, 只要到达过程就结束了, 所以我们不会去更新它的价值分

价值迭代: 第2轮

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

一定要注意, **return** 是从当前时刻到终止时刻的奖励总和, 因此只有过程结束后才能得到一个确定的数值。

考察这个格子, 它可以:

- 1) 向左走1步, reward为-1, 到达格子的价值打分为-1, return为-2;
- 2) 向右走1步, reward为-1, 到达奶酪格子价值是0, 则return为-1;

根据贪婪策略我们一定会往右走到奶酪格子(因为回报最大), 则当前格子的价值函数还是-1.

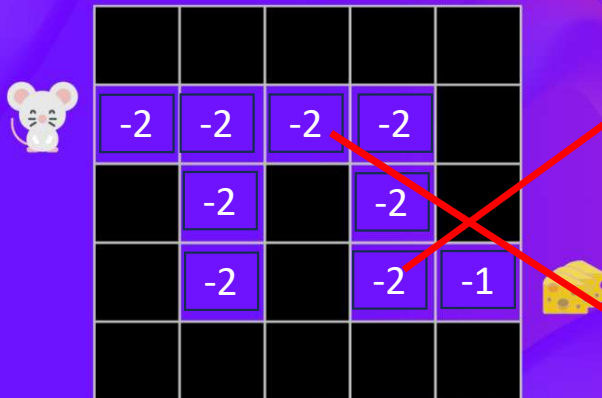
考察任意其它的格子, 它们往哪走的回报都是到达格子价值函数-1 + 移动 reward -1

所以此轮迭代会使价值分都为-2

价值迭代: 第3轮

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

考察这个格子，它可以：

- 1) 向上走1步，reward为-1，到达格子的价值打分为-2，return为-3；
- 2) 向右走1步，reward为-1，到达价值是-1，则return为-2；

根据贪婪策略我们一定会往右，则当前格子的价值函数还是-2.

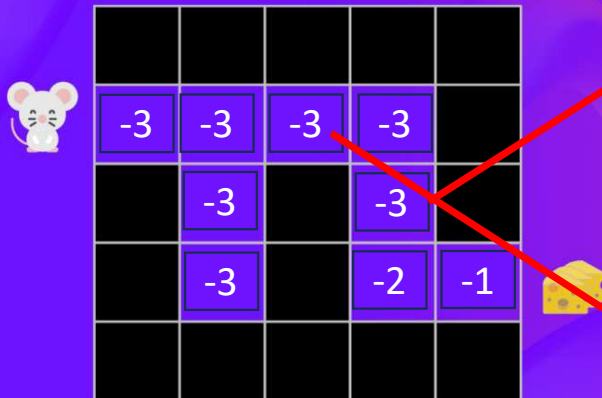
考察任意其它的格子，此轮迭代会使其价值函数为-3

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

价值迭代: 第4轮

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value function Expected discounted return Starting at state s

考察这个格子，它可以：

- 1) 向上走1步，reward为-1，到达格子的价值打分为-3，return为-4；
- 2) 向下走1步，reward为-1，到达价值是-2，则return为-3；

根据贪婪策略我们一定会往右，则当前格子的价值函数还是-3.

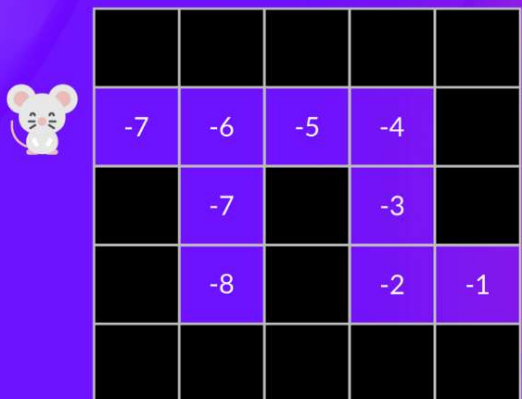
考察任意其它的格子，此轮迭代会使其价值函数为-4

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

价值迭代: 第N轮

The State Value Function

State Value Function: calculate the value of a state.



$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value
function

Expected discounted return

Starting
at state s

一定要注意，**return** 是从当前时刻到终止时刻的奖励总和，因此只有过程结束后才能得到一个确定的数值。

相信到这里我们已经能看出最后的结果了

会和我们上一次在上帝视角遍历所有可能得到的价值函数一致

这就是基于贝尔曼方程的价值迭代法

More on Bellman equation

<https://hrl.boyuai.com/chapter/1/马尔可夫决策过程>

在马尔可夫奖励过程中，一个状态的期望回报（即从这个状态出发的未来累积奖励的期望）被称为这个状态的**价值**（value）。所有状态的价值就组成了**价值函数**（value function），价值函数的输入为某个状态，输出为这个状态的价值。我们将价值函数写成

$V(s) = \mathbb{E}[G_t | S_t = s]$ ，展开为

$$\begin{aligned} V(s) &= \mathbb{E}[G_t | S_t = s] \\ &= \mathbb{E}[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots | S_t = s] \\ &= \mathbb{E}[R_t + \gamma(R_{t+1} + \gamma R_{t+2} + \dots) | S_t = s] \\ &= \mathbb{E}[R_t + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}[R_t + \gamma V(S_{t+1}) | S_t = s] \end{aligned}$$

在上式的最后一个等号中，一方面，即时奖励的期望正是奖励函数的输出，即 $\mathbb{E}[R_t | S_t = s] = r(s)$ ；另一方面，等式中剩余部分 $\mathbb{E}[\gamma V(S_{t+1}) | S_t = s]$ 可以根据从状态 s 出发的转移概率得到，即可以得到

$$V(s) = r(s) + \gamma \sum_{s' \in S} p(s' | s) V(s')$$

上式就是马尔可夫奖励过程中非常有名的**贝尔曼方程**（Bellman equation），对每一个状态都成立。若一个马尔可夫奖励过程一共有 n 个状态，即 $S = \{s_1, s_2, \dots, s_n\}$ ，我们将所有状态的价值表示成一个列向量 $\mathcal{V} = [V(s_1), V(s_2), \dots, V(s_n)]^T$ ，同理，将奖励函数写成一个列向量 $\mathcal{R} = [r(s_1), r(s_2), \dots, r(s_n)]^T$ 。于是我们可以将贝尔曼方程写成矩阵的形式：

$$\begin{aligned} \mathcal{V} &= \mathcal{R} + \gamma P \mathcal{V} \\ \begin{bmatrix} V(s_1) \\ V(s_2) \\ \dots \\ V(s_n) \end{bmatrix} &= \begin{bmatrix} r(s_1) \\ r(s_2) \\ \dots \\ r(s_n) \end{bmatrix} + \gamma \begin{bmatrix} P(s_1|s_1) & P(s_2|s_1) & \dots & P(s_n|s_1) \\ P(s_1|s_2) & P(s_2|s_2) & \dots & P(s_n|s_2) \\ \dots & \dots & \dots & \dots \\ P(s_1|s_n) & P(s_2|s_n) & \dots & P(s_n|s_n) \end{bmatrix} \begin{bmatrix} V(s_1) \\ V(s_2) \\ \dots \\ V(s_n) \end{bmatrix} \end{aligned}$$

More on Bellman equation

<https://www.bilibili.com/video/BV1sd4y167NS>

【强化学习的数学原理】课程：从零开始到透彻理解（完结）

182.0万 1.8万 2022-09-16 19:58:06 未经授权，禁止转载

Shiyu Zhao's course on reinforcement learning

Lecture 2: Bellman Equation

Part 1: Motivating examples – why is return important? (16min)

Part 2: State value (7min)

Part 3: Bellman equation – Derivation (15 min)

Part 4: Bellman equation – Matrix-vector form and solution (13 min)

Part 5: Action value (10 min)